

Sofia Lima dos Santos
Paradigmas da Programação - Lista 5

Exercício 1:

-A expressão do lado esquerdo é avaliada na seguinte ordem:

```
1 == 4 --> False
False && True --> False
```

-A expressão do lado direito é avaliada na ordem:

```
(4*8) --> 32 (multiplicação)
mod 32 31 --> 1 (divisão inteira)
1^2-5 --> -4 (exponenciação e subtração)
```

Logo, o resultado é (False, -4)

Exercício 2:

Podemos citar as expressões $2^{(-3)}$ e $2^{**}(-3)$ que se comportam de maneiras diferentes devido aos operadores:

```
ghci> 2 ** (-3)
0.125
ghci> 2^(-3)
*** Exception: Negative exponent
```

O erro ocorre pois o operador "^" aceita expoentes inteiros mas apenas não negativos.

Exercício 3:

```
dobro:: Double -> Double
dobro x = 2 * x
```

Exercício 4:

```
incremento:: Double -> Double
decremento:: Double -> Double
```

```
incremento x = x + 1
decremento y = y - 1
```

Exercício 5:

":t" é usado para retornar o tipo da expressão. Nesse caso, a classe de tipo da função foi inferida já que nenhum tipo foi explicitamente informado.

Poderíamos definir os tipos:

```
incremento :: Double -> Double
```

```
decremento :: Double -> Double
```

e será retornado:

```
decremento(incremento 9) :: Double
```

Exercício 6:

```
sobeDesce x y = (x + 1, y - 1)
```

Exercício 7:

```
incremento x = x + 1
```

```
decremento y = y - 1
```

```
sobeDesce x y = (incremento x, decremento y)
```

Exercício 8:

```
trocaValor (x,y) = (y,x)
```

Exercício 9:

```
-- Definindo um número
```

```
n :: Int
```

```
n = -531
```

```
-- usando a função negate para mudar o sinal do número
```

```
n_negado :: Int
```

```
n_negado = negate n
```

Exercício 10:

O erro é devido ao argumento "-8" que não está sendo entendido como um número.

Para corrigir é necessário colocá-lo entre parênteses:

```
ghci> negate (-8)
```

```
8
```